

Idempotency

Handling — Complete Study Notes

Safe Retries · Idempotency Keys · UUID
Critical Section · Distributed Locking · HTTP 409

TOPICS COVERED

1. Idempotency vs Concurrency

2. Why POST is Non-Idempotent

3. Idempotency Key (UUID)

4. Server Handling Flow

5. Parallel Requests & Race Conditions

6. Critical Section & Mutex

7. Distributed Locking with Redis

8. Quick Revision & Interview Questions

Table of Contents

1. What is Idempotency?	3
2. Idempotency vs Concurrency	3
3. Why POST is Not Idempotent	3
4. The Idempotency Key (UUID)	4
5. Server Handling Flow	4
6. Request Status Lifecycle	5
7. Parallel Requests & Race Conditions	5
8. Critical Section & Distributed Locking	6
9. End-to-End Flow Diagram	6
10. Quick Revision & Interview Questions	7

1. What is Idempotency?

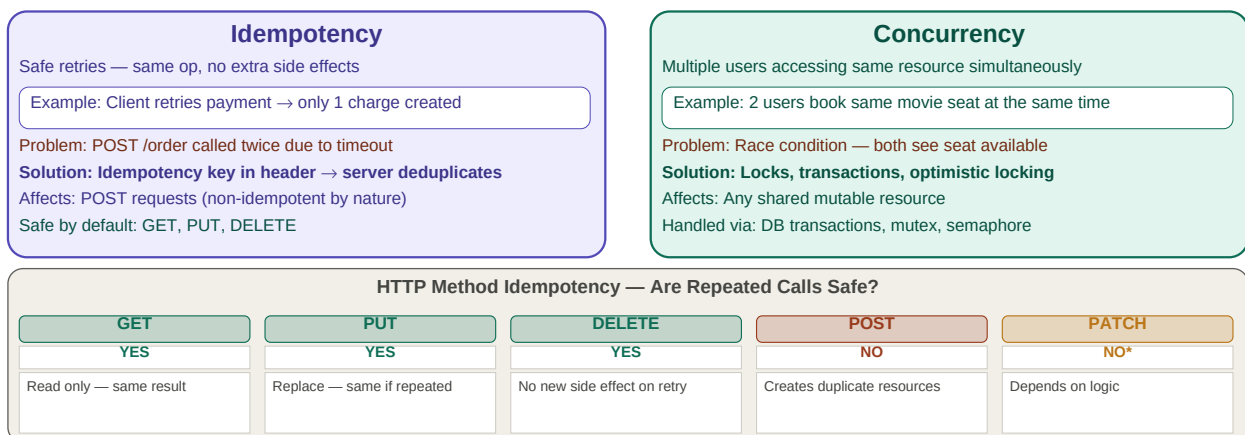
Idempotency is the property that allows a client to safely **retry the same operation multiple times** without causing unintended side effects. No matter how many times you call an idempotent operation, the result is the same as calling it once.

Why it matters: Networks are unreliable. A client sends a POST /payment request. The server processes it and charges the customer — but the response times out. The client has no idea if it succeeded. Should it retry? Without idempotency: retrying creates a second charge. With idempotency: retrying returns the original result safely.

Without Idempotency	With Idempotency
Client retries POST → duplicate order created	Client retries POST → server deduplicates using key
Payment charged twice	Payment charged once, retry returns same receipt
Cart item added twice	Cart item added once, retry returns existing item
Server has no memory of previous request	Server checks idempotency key in DB/cache

2. Idempotency vs Concurrency

Idempotency vs Concurrency — Core Concepts



Idempotency: safe retries via deduplication · Concurrency: shared resource conflicts · GET/PUT/DELETE safe by default · POST needs explicit handling

3. Why POST is Not Idempotent

POST requests create new resources. Every POST call to /orders creates a new order. This is by design — but it means duplicate POST calls (from retries or client bugs) cause duplicate resources. You cannot "fix" this at the HTTP level; you must handle it explicitly in your server logic.

Common causes of duplicate POST requests

- Network timeout: server processes request but client never receives response → client retries
- User double-clicks a button (rapid successive calls)
- Same user logged in on two tabs/devices simultaneously

- Mobile app retries after brief connectivity loss
- Message queue re-delivery (at-least-once delivery guarantee)

GET /orders/123 called twice → same order returned. No side effect. Safe.

PUT /orders/123 {status: shipped} called twice → still shows "shipped". No duplicate. Safe.

DELETE /orders/123 called twice → first deletes it, second returns 404. No duplicate. Safe.

POST /orders called twice → two separate orders created! NOT safe without idempotency handling.

4. The Idempotency Key

An **idempotency key** is a unique identifier that the **client generates** for each logical operation and sends in the request header. The server uses this key to recognise and deduplicate repeated requests.

Property	Details
Who generates it	The client — before sending the request
What it is	UUID v4 (universally unique identifier) — e.g. 550e8400-e29b-41d4-a716-446655440000
Where it goes	Request header: Idempotency-Key:
What it covers	One logical operation (e.g. "place this specific order")
Scope	Per user, per operation type, per attempt
Expiry	Server stores key for a configurable window (e.g. 24 hours)
If missing	Server returns 400 Bad Request

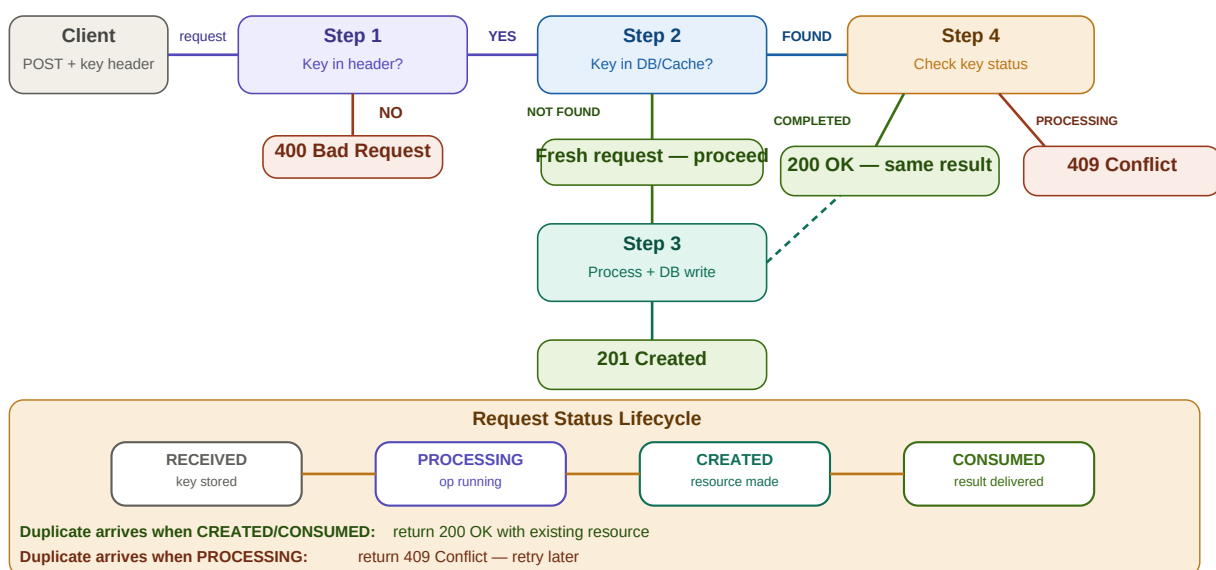
Key format: UUID v4 is sufficient. Optionally append: {userId}:{operationType}:{timestamp} for more uniqueness.

Real-world: Stripe uses "Idempotency-Key" header. PayPal uses "PayPal-Request-Id". Both are UUIDs.

The key must be unique per intent — if a user wants to place a new order, generate a new UUID. Same UUID = same operation.

5 & 6. Server Handling Flow & Status Lifecycle

Server Handling Flow — POST Request with Idempotency Key



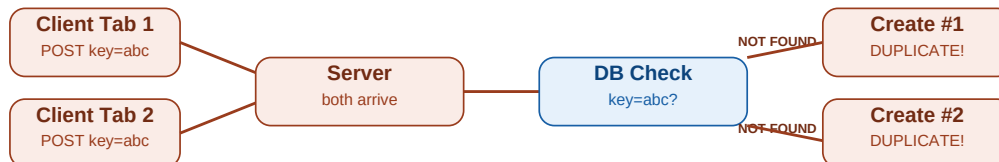
5-step server flow: validate key → DB lookup → process or deduplicate → status lifecycle:
RECEIVED→PROCESSING→CREATED→CONSUMED

7 & 8. Parallel Requests — Race Condition & Critical Section

Sequential duplicates are handled by the DB lookup — second request finds the key and returns 200/409. But **parallel duplicates** — two requests arriving at nearly the same instant — both query the DB simultaneously, both find nothing, and both create resources. This is a **race condition**.

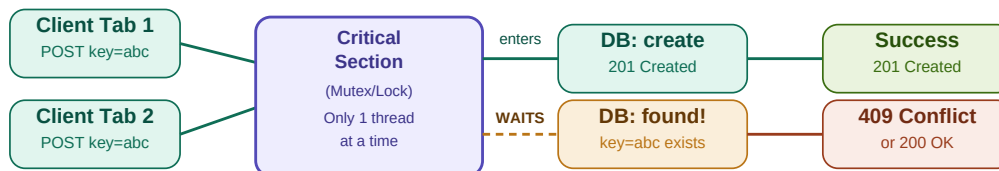
Parallel Duplicate Requests — Race Condition Solution

Without Critical Section (PROBLEM)



Both requests find no existing key → both create resource → 2x orders/payments!

With Critical Section (SOLUTION)



Distributed systems: Use Redis SETNX (set-if-not-exists) as a distributed lock instead of in-process mutex. Redis is ms-fast vs DB locks (minutes). Ensures atomic check+lock across all servers.

Without critical section: both parallel requests create resource (race) · With mutex/Redis lock: only one proceeds, other gets 409

Distributed Locking with Redis

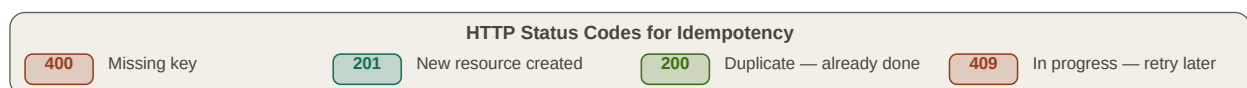
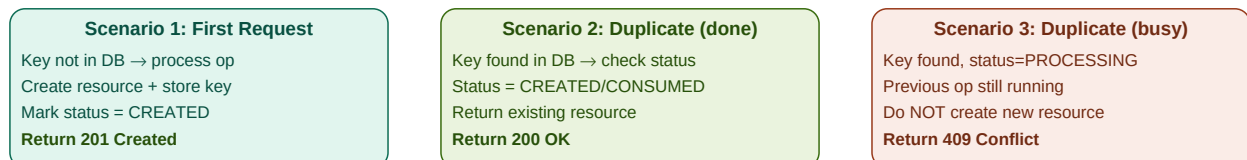
- Single server: use mutex (in-process lock) — only one thread processes per idempotency key at a time
- Multiple servers: in-process mutex is insufficient — different servers have different locks
- Solution: Redis SETNX ("set if not exists") — atomic check+set across all servers
- Redis SET key NX PX 5000: sets key only if not exists, expires in 5 seconds
- Redis is ms-fast vs DB row locks (which can be slow). Preferred for distributed rate limiting.
- If Redis SETNX succeeds: this server owns the key → process the request
- If Redis SETNX fails: another server is processing it → return 409 Conflict

9. End-to-End Flow

Complete Idempotency Flow — End to End



Three Scenarios



6-step E2E: client generates UUID → POST with header → validate → DB/Redis check → process → respond with 201/200/409

HTTP Code	When returned	Meaning
201 Created	First time — new resource created	Success — client can safely use the resource
200 OK	Duplicate — previous operation completed	Idempotent — return same result as before
409 Conflict	Duplicate — previous still processing	Race condition — client should retry after delay
400 Bad Request	Idempotency-Key header missing	Client error — must include key in header

10. Quick Revision & Interview Questions

Quick Revision

- > Idempotency = safe retries. Same operation called N times = same result as calling once.
- > Concurrency = multiple users accessing same resource simultaneously (different problem).
- > GET, PUT, DELETE are idempotent by nature. POST is NOT — it creates resources on every call.
- > Causes of duplicate POST: network timeouts, double-click, multi-tab, mobile retry, queue redelivery.
- > Solution: Idempotency Key — UUID generated by client, sent in Idempotency-Key request header.
- > Server flow: 1. Key present? → 400 if not. 2. Key in DB? → 3. First time: process + 201. 4. Done: 200. Processing: 409.
- > Status lifecycle: RECEIVED → PROCESSING → CREATED → CONSUMED.
- > Sequential duplicate: DB lookup handles it — second request finds key → returns 200 or 409.
- > Parallel duplicate: race condition — both find nothing → both create. Solution: critical section.
- > Single server: in-process mutex (only 1 thread per key at a time).
- > Distributed: Redis SETNX — atomic check+set across all servers. ms-fast vs DB locks.
- > Redis SET key NX PX 5000 — sets only if not exists, auto-expires in 5s to prevent deadlocks.

- > Used by: Stripe (Idempotency-Key header), PayPal (PayPal-Request-Id), all payment APIs.
- > Interview tip: Always distinguish idempotency (retries) from concurrency (simultaneous access).

Interview Questions

- 1. What is idempotency? How is it different from concurrency?
- 2. Why is POST not idempotent? Which HTTP methods are?
- 3. What causes duplicate POST requests in production systems?
- 4. Explain how an idempotency key works end-to-end.
- 5. What should the server return when a duplicate request arrives and the original is still processing?
- 6. Walk me through the status lifecycle of an idempotent request.
- 7. What is a race condition in the context of idempotency? How do you solve it?
- 8. Why is in-process mutex insufficient for distributed idempotency handling?
- 9. How does Redis SETNX solve the distributed idempotency race condition?
- 10. Design an idempotent payment API. What headers, DB schema, and server logic would you use?
- 11. What happens if the Redis lock expires before the operation completes?
- 12. How long should an idempotency key be stored on the server?

Key Terms

Idempotency	Same operation called N times produces same result as calling once
Idempotency Key	UUID sent by client in request header to identify a unique logical operation
UUID v4	Universally Unique Identifier — 128-bit random key, practically collision-free
Duplicate Request	Same POST sent multiple times due to timeout, retry, or client bug
Race Condition	Two parallel requests both see no existing key and both create resources
Critical Section	Code region where only one thread may execute at a time — prevents race
Mutex	Mutual exclusion lock — single-server solution for critical section
Redis SETNX	SET if Not eXists — atomic Redis command used as distributed lock
409 Conflict	HTTP status: duplicate request received while original is still processing
Status: CREATED	Request was processed and resource was created
Status: CONSUMED	Result has been returned to client — operation fully complete
Distributed Lock	Lock that works across multiple servers via shared store (Redis)
At-least-once	Message delivery guarantee that may cause duplicate messages — needs idempotency